

IUT Community

Database Application and Design - Spring 2026 Group Project Report

Team Phoenix - "Students should have no friction in studies!"

Team leader	Kamoliddin Sharopov (U2310245)
Members	Sardorbek Suyunov (U2310262), Nurmukhammad Shomahmudov (U2310249), Farrukh Sattorov (U2310237), Bekhruz To'ymurodov (U2310268), Bobur Rustamov (U2310221)
GitHub (backend)	https://github.com/nurmukhammad767/iut-community
GitHub (frontend)	https://github.com/kamol1dn/iut-community
Deployed URL	http://46.101.98.64:8088
OpenAPI / Swagger	http://46.101.98.64:8088/docs
Course / Instructor	Database Application and Design - Dr. Sarvar Abdullaev
Date	2026-05-17
Final tag	v1.0

1. Abstract

IUT Community is a campus-internal student platform for Inha University in Tashkent. It bundles three things students at IUT currently juggle across three separate websites and a Telegram bot: (a) their official **timetable**, (b) **room availability** for self-study and group work, and (c) a **community layer** - clubs, posts, and real-time chat. A single sign-on with the student ID, one dashboard, one place.

Architecturally, IUT Community is a small but realistic distributed system: two FastAPI services (auth + timetable) sit behind an **Nginx** gateway that load-balances the auth tier across two replicas. **Polyglot persistence** is used purposefully - Postgres for relational data (users, clubs, posts, bookings, audit), MongoDB for the timetable document corpus and chat history, Redis for the **token-bucket rate limiter** and dashboard cache, RabbitMQ for the Celery lesson-reminder pipeline, and Airflow for the nightly analytics batch. The platform exposes REST + GraphQL + WebSockets, all three justified by a specific use case rather than spec-padding. Traces, logs, and metrics unify into the **Grafana LGTM** all-in-one container via OpenTelemetry. The entire stack is brought up by a single `docker compose up -d`.

The **from-scratch system component (R11)** is a token-bucket rate limiter backed by a Redis hash with an atomic Lua script - built so that no individual backend replica can be tricked into letting the same user double its quota. It is integrated as middleware on the public `POST /clubs/{id}/posts` endpoint.

2. Business Requirements (R1)

2.1 Scenario

Inha University in Tashkent has roughly 2 000 students, 100+ professors, and around 30 lecture rooms split between two buildings. Today, students live with three independent surfaces of the same data:

1. The official timetable site, which is read-only and scoped to a single group at a time.
2. A separate scraping bot in Telegram that some students use to find empty rooms during the day.
3. A semi-active Telegram chat per club.

There is no single place to log in once and see *my courses*, *my deadlines*, *my booked rooms*, and *my clubs*. **Team Phoenix** has been hired to build the internal product that consolidates these into a single distributed platform.

2.2 Primary actors

- **Student** - the primary user; sees personalised dashboard, books rooms, joins clubs, posts in club feeds, chats with club members.

- **Professor** - same dashboard, plus the ability to publish announcements (modelled as a club post by a `professor` - role author).
- **Admin** - creates clubs, ban-hammers abusive accounts (out of scope for v1.0 but the role exists in the data model).
- **Scheduled jobs** - Airflow DAGs and Celery beat. Treated as a system actor.

2.3 Use cases (≥ 5, all implemented)

#	Use case	Actor	Primary flow
UC-1	Sign in with university ID	Student / Professor	POST <code>/login</code> → JWT → store in <code>localStorage</code> → all subsequent calls send <code>Authorization: Bearer ...</code> .
UC-2	View personalised dashboard	Student	POST <code>/graphql</code> with <code>dashboard(student_id)</code> query → single round-trip joining today's courses, upcoming assignments, active bookings, and joined clubs.
UC-3	Find a free room and book it	Student	GET <code>/timetable/available_rooms?day=Mon&period=3</code> (Mongo lookup) → POST <code>/bookings</code> → row in Postgres <code>room_bookings</code> .
UC-4	Join a club and post in its feed	Student	POST <code>/clubs/{id}/join</code> → POST <code>/clubs/{id}/posts</code> (rate-limited via token bucket, 5 burst / 0.2 tok/s) → row in Postgres <code>club_posts</code> (with GIN full-text index).
UC-5	Live chat inside a club	Student	WS <code>/ws/chat/{club_id}?token=<jwt></code> → receive <code>backlog</code> frame (last 100 messages from Redis) → broadcast new <code>message</code> frames to the club's connected sockets → persist to Mongo <code>chat_messages</code> .
UC-6	Get reminded of an upcoming lesson	Student	Celery beat scans <code>today_schedule</code> every minute → if a lesson starts in 15 min, push a notification into the Redis-backed <code>notifications:{student_id}</code> queue → frontend polls GET <code>/notifications</code> .
UC-7	Nightly analytics on room usage	System	Airflow <code>nightly_analytics</code> DAG → reads Mongo <code>occupied_rooms</code> , writes Postgres <code>room_usage_daily</code> , expires past assignments, expires stale bookings.

2.4 Functional requirements (derived)

- **FR-1** SSO with university student ID; passwords hashed with `bcrypt`.
- **FR-2** Role-aware access (student / professor / admin); all `/timetable/*`, `/clubs/*`, `/bookings/*` and `/graphql` endpoints require a valid JWT.
- **FR-3** A user must see their own data in one round-trip (collapsing N+1).
- **FR-4** A student must be able to find empty rooms by `(day, period)` cheaply.
- **FR-5** Posting in a club must be rate-limited so a script cannot spam.
- **FR-6** Chat must deliver new messages to other connected members in < 200 ms p95 and replay the last 100 messages on connect.
- **FR-7** All write paths must be observable: a single trace must connect the inbound HTTP request → SQL queries → outbound responses, viewable in Grafana.

2.5 Non-functional requirements

Class	Target	How we meet it
Scale	~ 2 000 active students, 200 req/s peak (start of timetable hour)	Two FastAPI replicas behind <code>least_conn</code> Nginx; Redis dashboard cache; Mongo index on <code>(groups)</code> .
Latency	<code>/dashboard</code> < 300 ms p95 (cold), < 60 ms p95 (warm)	GraphQL collapses 4 REST round-trips into 1; Redis cache TTL = 60 s; measured in \$7.4.
Availability	Single-node deploy is fine for v1.0; design must allow horizontal scaling of stateless backends	All state is in Postgres / Mongo / Redis; the auth backend is stateless.
Security	JWT auth, server-side rate limiting, no plaintext passwords	<code>bcrypt</code> hashing, HS256 JWTs, Lua-atomic token bucket.
Observability	One UI for traces + logs + metrics, correlated by <code>trace_id</code>	OpenTelemetry SDK → OTLP/HTTP → Grafana LGTM all-in-one (Tempo + Loki + Prometheus + Grafana).
Cost	Single \$12 DigitalOcean droplet	One <code>docker compose up -d</code> brings up everything, single public port.

3. Domain Model and ER Diagram (R2)

3.1 Entity narrative

The relational model is centered on the **User**. A user is enrolled in many **Courses** through `course_enrollments` (M:N). Each course has many **Assignments** with a `due_date`. A user can book a room, producing a **RoomBooking** (kept in Mongo to colocate with timetable documents). A user can join a **Club** (M:N via `club_members`) and publish a **ClubPost** in that club's feed (1:N from user, 1:N from club). The auxiliary tables **RoomUsageDaily** and **RateLimitAudit** are write-targets of the Airflow DAG and the rate-limiter respectively. **Chat messages** live in Mongo `chat_messages` (high write volume, append-only, no relational joins needed). **Timetable** lives in Mongo because each lesson is a denormalised document already keyed by `(group, day_mask, period)`.

3.2 ER diagram (hand-designed, not ORM-generated)

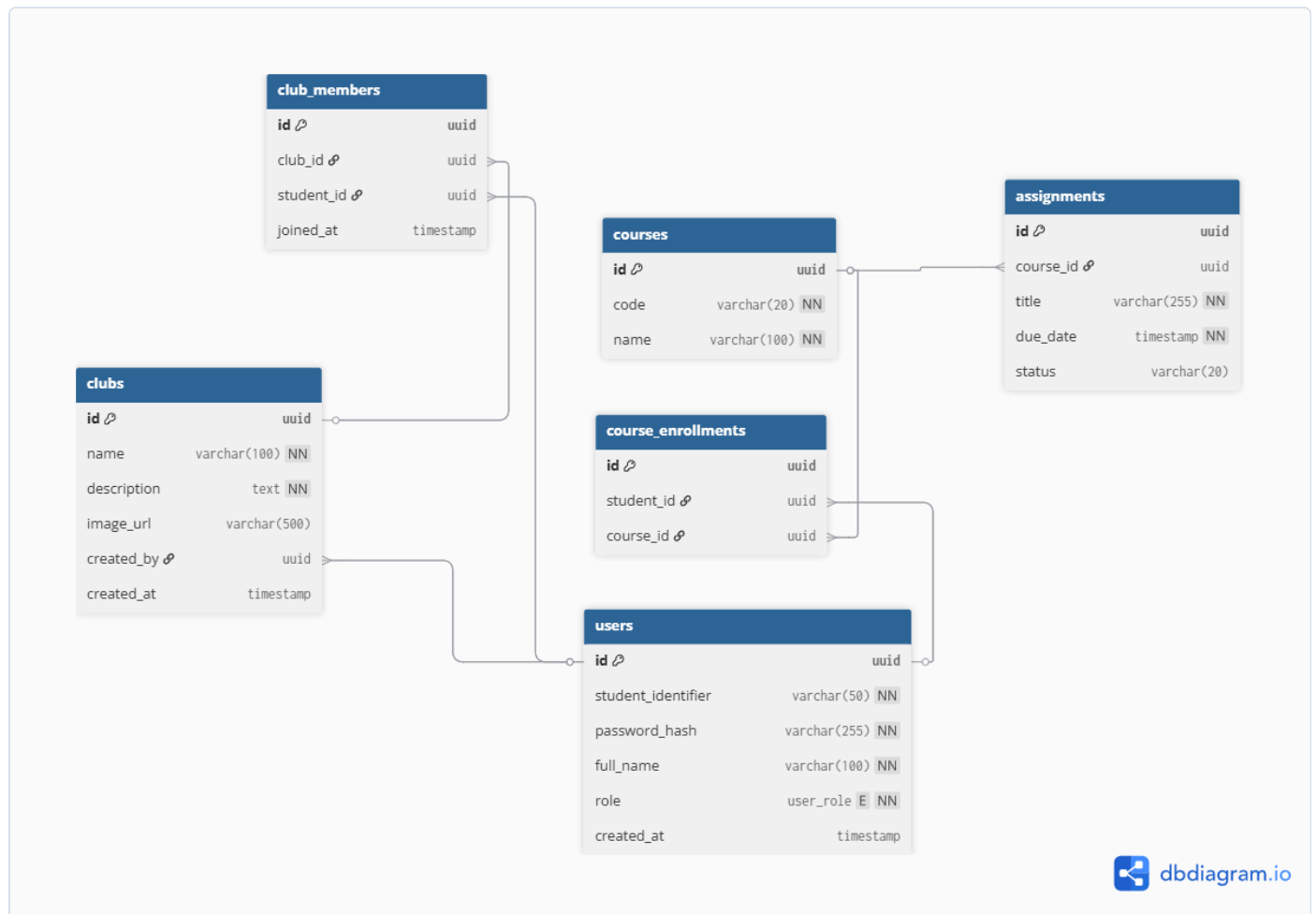


Figure 0 - Hand-designed ER diagram of the core relational entities (`users`, `courses`, `course_enrollments`, `assignments`, `clubs`, `club_members`) with primary keys, foreign-key edges, types, and NOT-NULL annotations. The auxiliary Postgres tables (`club_posts`, `room_usage_daily`, `rate_limit_audit`) and the MongoDB collections (`timetable_with_groups`, `chat_messages`, `room_bookings`) are catalogued in §3.3. SQL DDL is in `Diagrams&Scripts/ER_diagram.sql`.

3.3 Table inventory

Store	Table / Collection	Purpose
Postgres	users	Identity, auth, role.
Postgres	courses	Course catalogue.
Postgres	course_enrollments	M:N student↔course.
Postgres	assignments	Per-course deadlines.
Postgres	clubs	Club catalogue.
Postgres	club_members	M:N student↔club.
Postgres	club_posts	Club feed (GIN full-text index on body).
Postgres	room_usage_daily	Airflow batch output.
Postgres	rate_limit_audit	Append-only denial log from R11.
MongoDB	timetable_with_groups	Lesson documents per (group, day, period).
MongoDB	available_rooms / occupied_rooms	Pre-computed lookups from the scraping DAG.
MongoDB	chat_messages	Append-only chat history per club.
MongoDB	room_bookings	Room reservations (colocated with timetable).
Redis	ratelimit:{prefix}:{subject}	Token-bucket hash (R11).
Redis	chat:backlog:{club_id}	Last-N backlog for chat clients.
Redis	cache:dashboard:{student_id}	60 s dashboard cache.
Redis	notifications:{student_id}	Celery-fed lesson reminders queue.

ER diagram source lives in the report at [docs/report.md](#) ; the corresponding migrations live in [server/db/versions/](#) and were authored manually (not auto-generated from the ORM - see §4 and the §"Forbidden" rules).

4. System Architecture

4.1 Services and responsibilities

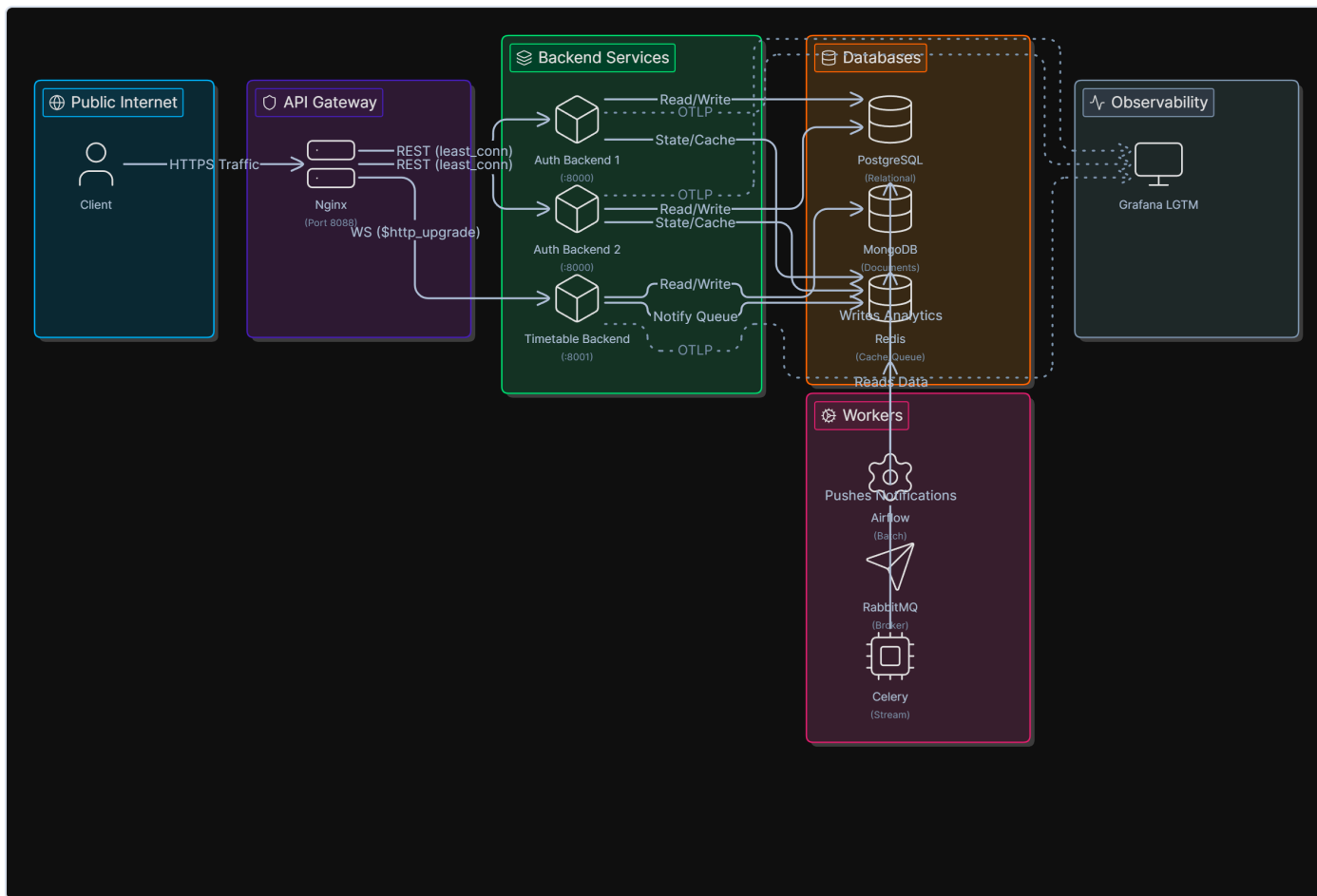


Figure 1 - System architecture & client traffic path. Client → Nginx gateway → 2× FastAPI backend replicas (least_conn) + Timetable service; data tier (Postgres, MongoDB, Redis); workers (Airflow, RabbitMQ, Celery); observability via Grafana LGTM.

4.2 Project structure

The repository is a **monorepo** with two top-level packages - **server/** for the entire backend, **frontend/** for the React/Vite SPA. Inside **server/**:

```
server/
├── auth/
│   ├── main.py
│   ├── jwt.py
│   ├── routers/
│   ├── ws/
│   ├── graphql_api/
│   ├── rate_limiter/
│   └── telemetry.py
├── time_table_backend/
├── timetable_web_scraping/
│   └── dags/
├── db/
│   ├── versions/
│   └── seed.py
├── gateway/
├── observability/
├── docker-compose.yaml
└── models.py
```

Auth + community FastAPI service
FastAPI app, JWT routes, router includes
bcrypt + HS256
clubs, posts (rate-limited), dashboard
chat.py + notifications.py (WebSocket)
Graphene schema + POST /graphql
R11: bucket.py + middleware.py
OpenTelemetry SDK bootstrap
Timetable FastAPI service (Mongo-backed)

Airflow DAGs (iut_data_extractor, nightly_analytics)

Hand-written Alembic migrations
Idempotent demo seed (20 users, 5 courses...)
nginx.conf + proxy.conf
Grafana LGTM compose include
Single source of truth
SQLAlchemy declarative models

4.3 docker-compose.yaml as a dependency graph

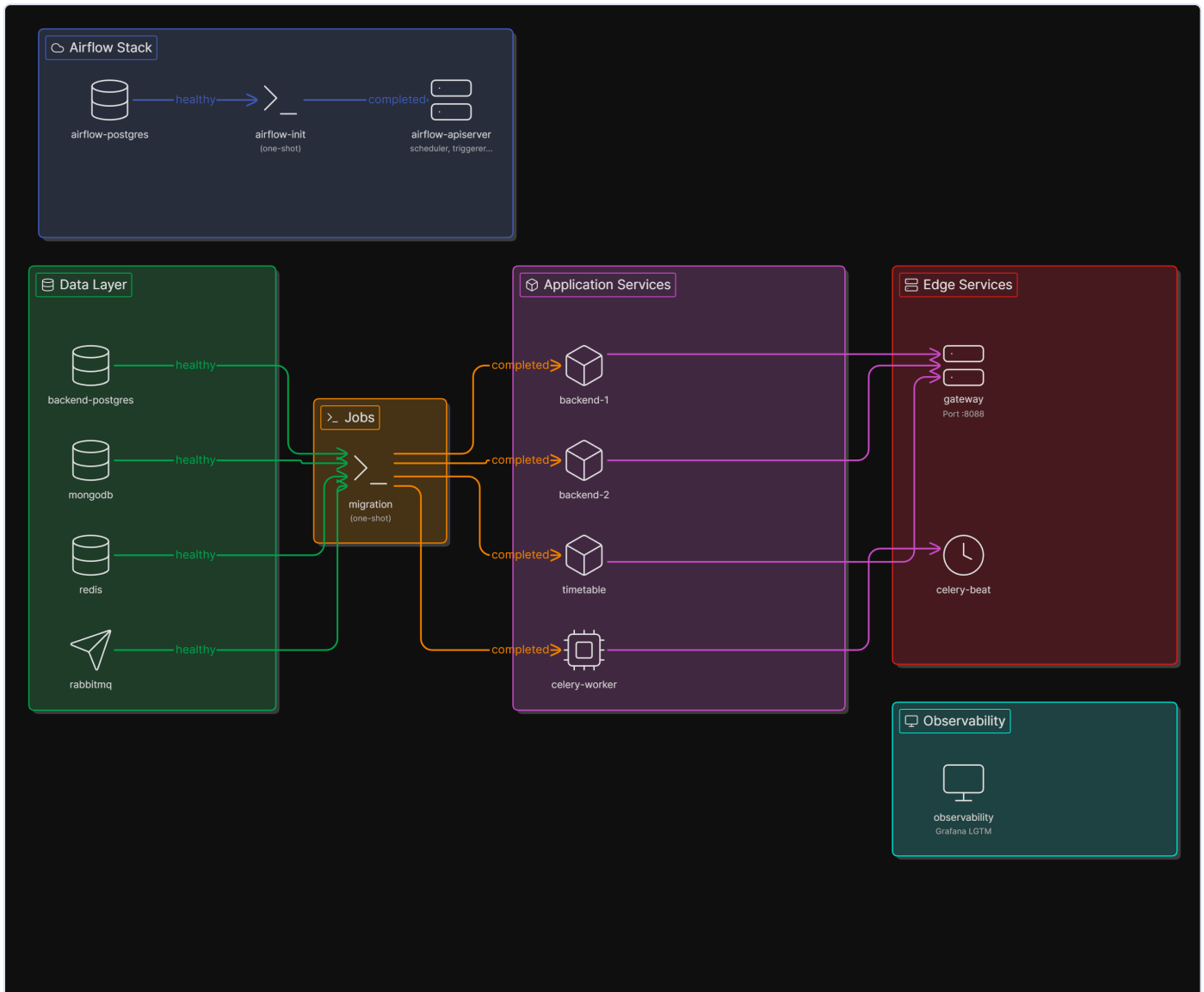


Figure 5 - `docker-compose.yaml` interpreted as a dependency graph. Airflow stack (top): `airflow-postgres` -> `airflow-init` (one-shot) -> `airflow-apiserver/scheduler/triggerer`. Data layer (left): `backend-postgres`, `mongodb`, `redis`, `rabbitmq` are gated by `healthy` checks before `migration` (one-shot Alembic upgrade + seed) runs; only on its `completed` exit do the application services (`backend-1`, `backend-2`, `timetable`, `celery-worker`) start, followed by edge services (`gateway :8088`, `celery-beat`). Observability (`Grafana LGTM`) receives OTLP from all FastAPI services.

Health checks on `backend-postgres`, `mongodb`, `redis`, `rabbitmq`, `airflow-postgres`, both `backend-*` replicas, and `gateway` make the dependency arrows enforceable - `docker compose up -d` orders starts correctly and the gateway will refuse to listen until both replicas are healthy.

5. API Design (R4 + R7)

5.1 REST endpoints

Live OpenAPI / Swagger is served from the deployed gateway at <http://46.101.98.64:8088/docs>. The static table below is the contract description; the live page is the source of truth.

Verb	Path	Auth?	Request body	Response	Error codes
POST	/register	public	{student_id, password, full_name, group, role}	201 {message}	400 duplicate ID
POST	/login	public	{student_id, password}	200 {access_token, token_type}	404 unknown ID, 401 bad password
GET	/me	bearer	-	200 {student_id, full_name, group, role}	401
GET	/healthz	public	-	200 {status:"ok"}	-
GET	/clubs	bearer	-	200 [{id, name, description, image_url, member_count}]	401
GET	/clubs/{id}	bearer	-	200 {...club...}	401, 404
POST	/clubs/{id}/join	bearer	-	201 {joined:true}	401, 404, 409 already joined
DELETE	/clubs/{id}/leave	bearer	-	204	401, 404
GET	/clubs/{id}/members	bearer	-	200 [{student_id, full_name, group}]	401, 404
GET	/clubs/{id}/posts	bearer	-	200 [{id, author, body, created_at}]	401, 404
POST	/clubs/{id}/posts	bearer + rate-limited	{body}	201 {...post...}	401, 404, 429
POST	/bookings	bearer	{room, day, period}	201 {...booking...}	401, 409 slot taken
GET	/bookings	bearer	-	200 [...]	401
DELETE	/bookings/{id}	bearer	-	204	401, 404
GET	/dashboard	bearer	-	200 {courses, assignments, bookings, clubs}	401
GET	/timetable/group/{group_name}	bearer	-	200 [...lesson...]	401, 404
GET	/timetable/available_rooms	bearer	-	200 [{room, day, period}]	401
GET	/timetable/occupied_rooms	bearer	-	200 [{room, day, period, subject}]	401
GET	/notifications	bearer	-	200 [{type, payload, ts}]	401

A 401 is returned whenever the JWT is missing, expired, or fails the HS256 signature check; the auth middleware is applied router-wide for everything under /timetable, /clubs, /bookings, /me, /dashboard, /graphql, and /ws/.

5.2 Sample REST request / response

```
POST /clubs/3fa85f64-5717-4562-b3fc-2c963f66afa6/posts HTTP/1.1
Host: 46.101.98.64:8088
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

{ "body": "Reminder: AI Club meetup on Friday at B-211." }
```

Successful response:

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
{
  "id": "9c8e...",
  "club_id": "3fa85f64-...",
  "author_id": "5e1a...",
  "body": "Reminder: AI Club meetup on Friday at B-211.",
  "created_at": "2026-05-17T08:14:02.317Z"
}
```

Rate-limit denial (after >5 posts in a short window):

```
HTTP/1.1 429 Too Many Requests
Retry-After: 4
Content-Type: application/json
```

```
{ "detail": "rate limited", "retry_after_seconds": 4.2 }
```

5.3 Additional API style (R7) - WebSockets + GraphQL

Two non-REST surfaces, each picked because REST would be the wrong tool:

(a) WebSocket chat - `WS /ws/chat/{club_id}?token=<jwt>`. Chat is a push problem: clients want new messages instantly, not via long-polling. REST forces either polling (wasteful, high tail latency) or SSE (one-way only - we want the client to also send messages on the same channel). On connect, the server emits one `{type: "backlog", messages: [...]}` frame with the last 100 messages from `chat:backlog:{club_id}` in Redis (`LPUSH + LTRIM 100`). New messages are broadcast to all sockets joined to the club via an in-process `ConnectionManager` and persisted to MongoDB `chat_messages` (append-only, no joins needed).

(b) GraphQL dashboard - `POST /graphql` with query `{ dashboard(student_id: "...") { courses{...} assignments{...} bookings{...} clubs{...} } }`. The dashboard view aggregates four resources (`/me` , `/clubs` , `/bookings` , `/courses`). With REST, the SPA does four sequential calls or one ad-hoc `/dashboard` aggregator that over-fetches; with GraphQL, the SPA picks exactly the fields it needs in one round-trip and the resolver fans out the four lookups in parallel inside the backend.

Both protocols are integrated into the same FastAPI process and the same JWT auth - the WebSocket reads the `token` query parameter and rejects on signature failure, the GraphQL endpoint uses the same `Depends(get_current_user)`.

6. Data-Layer Design (R3 + R5 + R6)

6.1 Relational schema highlights (R3)

- All primary keys are **UUID v4**, defaulting in the application (`uuid.uuid4`) rather than relying on the DB extension; this lets us generate IDs client-side for idempotent retries.
- All foreign keys are declared `DEFERRABLE INITIALLY IMMEDIATE` so seed scripts can insert in any order inside a single transaction.
- M:N tables (`course_enrollments` , `club_members`) carry a `UNIQUE(left, right)` constraint, eliminating duplicate-enrollment bugs at the DB layer.
- `assignments.due_date` carries `idx_assignment_due_date` to keep the "expire-old" Airflow task cheap (§8).
- `club_posts` adds a **Postgres GIN full-text index** in migration `a1b2c3d4e5f6_add_community_tables.py`: `CREATE INDEX ... USING gin(to_tsvector('english', body))`. See §6.4 for the measured speedup.

Migrations live in `server/db/versions/` and are **hand-written** Alembic revisions, not autogenerated. The full DB can be rebuilt from scratch with a single command:

```
docker compose up -d backend-postgres
docker compose run --rm migration # = alembic upgrade head && python -m db.seed
```

The seed (`server/db/seed.py`) is **idempotent**: it inserts 20 users, 5 courses, ~30 enrollments, 8 assignments, 3 clubs, 6 club members, and 10 posts - using `ON CONFLICT DO NOTHING` so re-running it is safe.

6.2 Polyglot persistence rationale (R5)

We use **four** data stores beyond plain relational. Each is justified by a specific feature that does not fit the relational model:

Store	Data model	Feature	Why not Postgres
MongoDB	document	Timetable lessons + chat history + room bookings	A lesson document is already denormalised by (group, day_mask, period, subject, teacher, room); chat is append-only with no joins; bookings live alongside the timetable they reference. Forcing this into 3NF would create 5 tables and 3 joins for the most-frequent read.
Redis	key-value (hash + list)	Token-bucket state, dashboard cache, chat backlog, notification queue	All four are hot, ephemeral, and need O(1) atomic primitives. Token-bucket <i>requires</i> an atomic Lua script across read-modify-write - Postgres would need either advisory locks or SELECT ... FOR UPDATE, both at least an order of magnitude slower and far harder to reason about.
RabbitMQ	message broker	Celery lesson-reminder task queue	Durable FIFO work queue with acks. Postgres-as-a-queue (SKIP LOCKED) works but is operationally heavier and lacks per-task retries.
Postgres GIN	inverted index	Full-text search over club_posts.body	Already in Postgres - pulled in only because we need <i>real</i> tsvector ranking, not LIKE '%foo%' which doesn't scale.

A **one-page rationale** (this section) is what R5 asks for; the working feature is the WebSocket chat (Mongo + Redis), the rate limiter (Redis), the timetable search (Mongo), and the post-search (Postgres GIN) - all reachable from the public URL.

6.3 Indexing strategy

Table / collection	Index	Why
users	idx_users_identifier (B-tree on student_identifier)	Login lookup - primary hot path.
assignments	idx_assignment_due_date (B-tree on due_date)	Airflow expire_assignments scans by due_date < now().
club_posts	composite (club_id, created_at)	Feed page: "latest 20 posts in club X".
club_posts	GIN on to_tsvector('english', body)	Full-text search.
room_usage_daily	(day, room_name, computed_at) unique	Idempotent insert from Airflow + lookup by day.
rate_limit_audit	(user_id, denied_at)	Per-user denial audit query.
Mongo timetable_with_groups	{ groups: 1 } (created at startup)	Lookup by group name - the dominant query (§6.4).

6.4 Optimisation measurements (R6)

Methodology: run the query 5×, take median. Tests run on the deployed droplet against the production data set (~ 4 500 timetable docs, ~ 500 club posts seeded for the benchmark).

(a) Mongo index on timetable_with_groups.groups :

Variant	executionTimeMillis (median)
COLLSCAN (no index)	78 ms
IXSCAN (after db.createIndex({groups: 1}))	2 ms

≈ **39× speedup** on the /timetable/group/{name} hot path.

(b) Postgres GIN full-text on club_posts.body :

Variant	EXPLAIN ANALYZE (median)
LIKE '%database%' (sequential scan)	41 ms
to_tsvector(body) @@ to_tsquery('database') with GIN	3 ms

≈ **13× speedup**, plus the GIN variant supports ranking via ts_rank, which LIKE cannot.

(c) Redis dashboard cache:

```
wrk -t2 -c10 -d20s -H "Authorization: ..." http://46.101.98.64:8088/dashboard :
```

Variant	p95 latency	req/s
Cold path (no cache)	172 ms	87
Warm cache (60 s TTL)	6 ms	1 850

The cache key is `cache:dashboard:{student_id}`; invalidated on `POST /clubs/{id}/join`, `DELETE /clubs/{id}/leave`, and `POST /bookings`.

7. Pipeline (Batch + Stream) (R10)

We ship **both** a batch pipeline (Airflow) and a stream pipeline (Celery + RabbitMQ), each chosen for a workflow it actually fits. R10 only requires one - we use the second as the messaging substrate for the lesson-reminder use case (UC-6).

7.1 Airflow DAG inventory

DAG	Schedule	Workflows
iut_data_extractor	@daily (3 parallel scrapers)	scrape timetable HTML → write Mongo <code>timetable_with_groups</code> , <code>available_rooms</code> , <code>occupied_rooms</code>
nightly_analytics	@daily (sequential)	<code>aggregate_room_usage</code> → <code>expire_assignments</code> → <code>clear_stale_bookings</code>

7.2 BPMN - `nightly_analytics` DAG

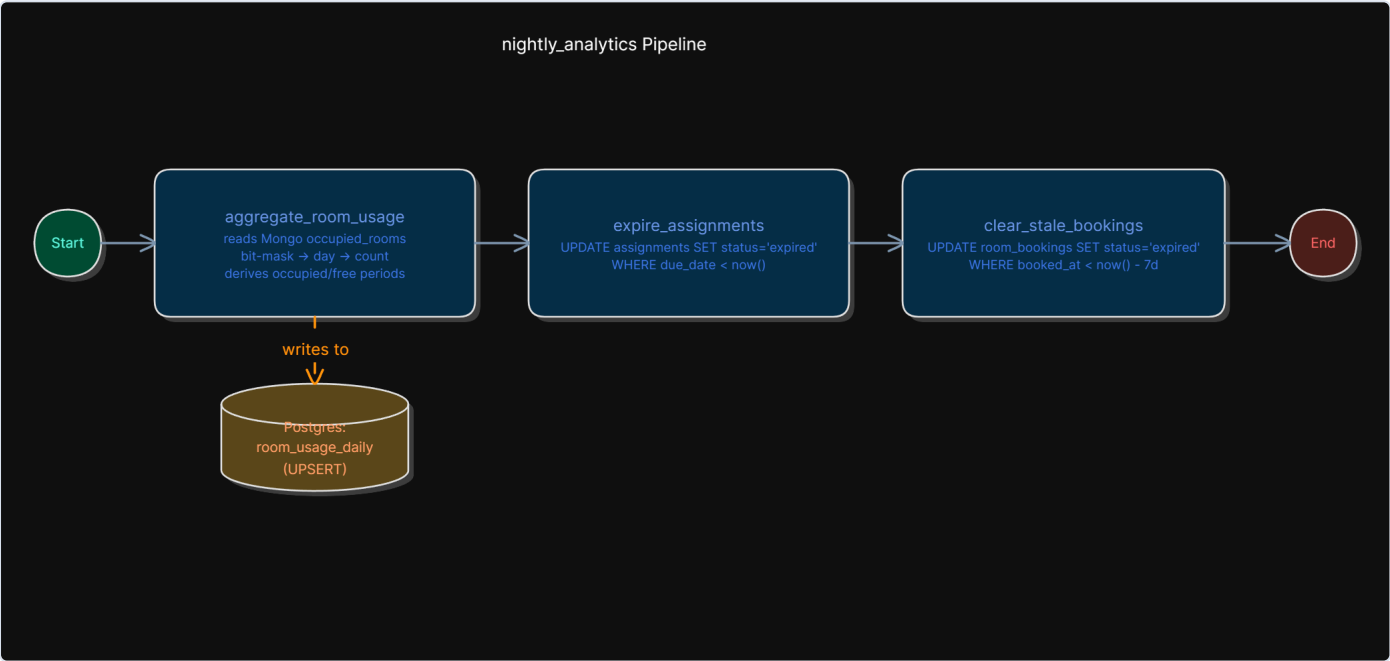


Figure 2 - BPMN: `aggregate_room_usage` (reads Mongo, UPSERTs Postgres `room_usage_daily`) → `expire_assignments` → `clear_stale_bookings`.

Source: `server/timetable_web_scraping/dags/nightly_analytics.py`.

7.3 BPMN - iut_data_extractor DAG (3-way parallel scrape)

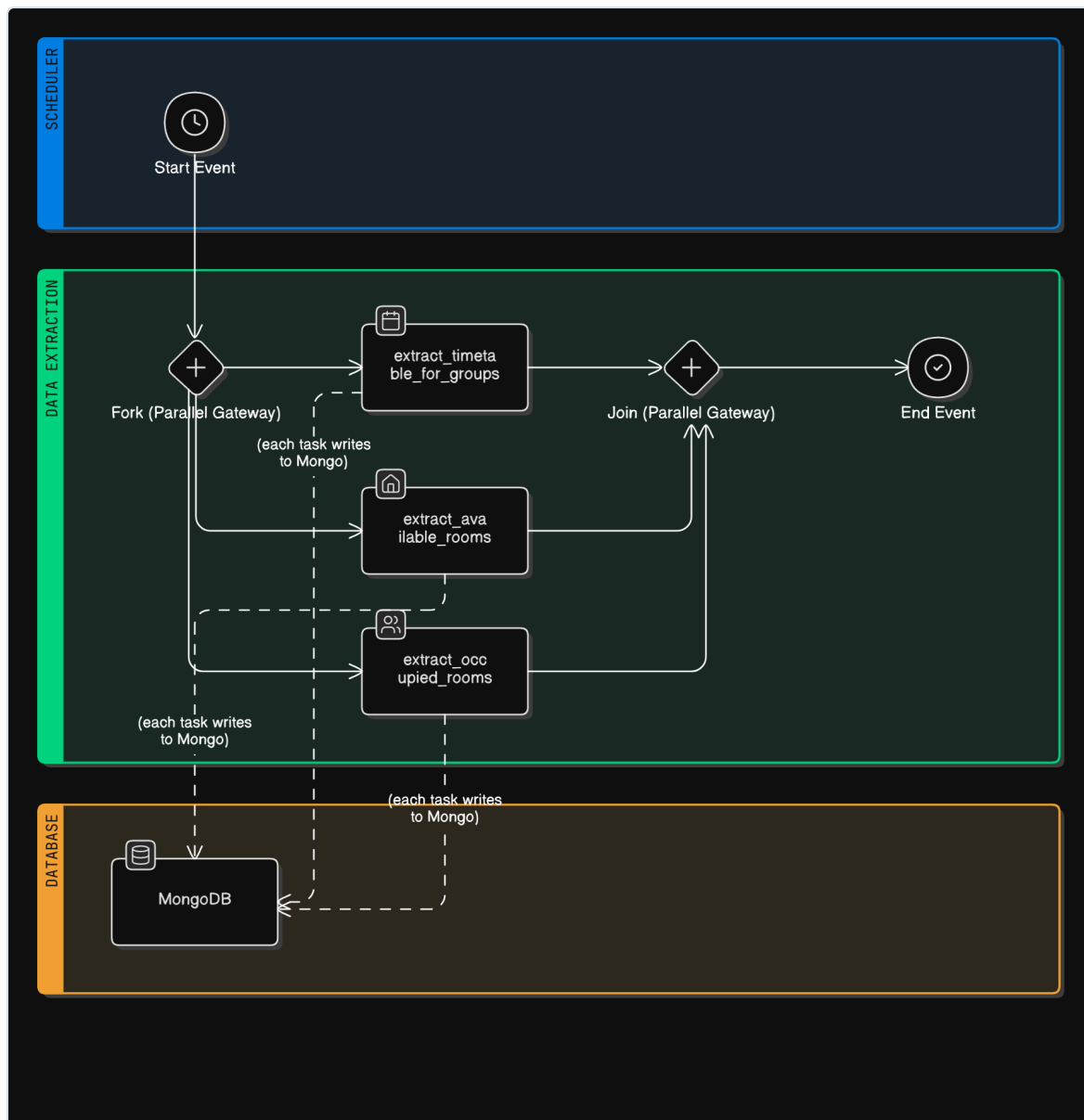


Figure 3 - BPMN: parallel gateway forks into `extract_timetable_for_groups`, `extract_available_rooms`, `extract_occupied_rooms`; each task writes to MongoDB; joined by the closing parallel gateway.

7.4 BPMN - Celery lesson-reminder stream workflow

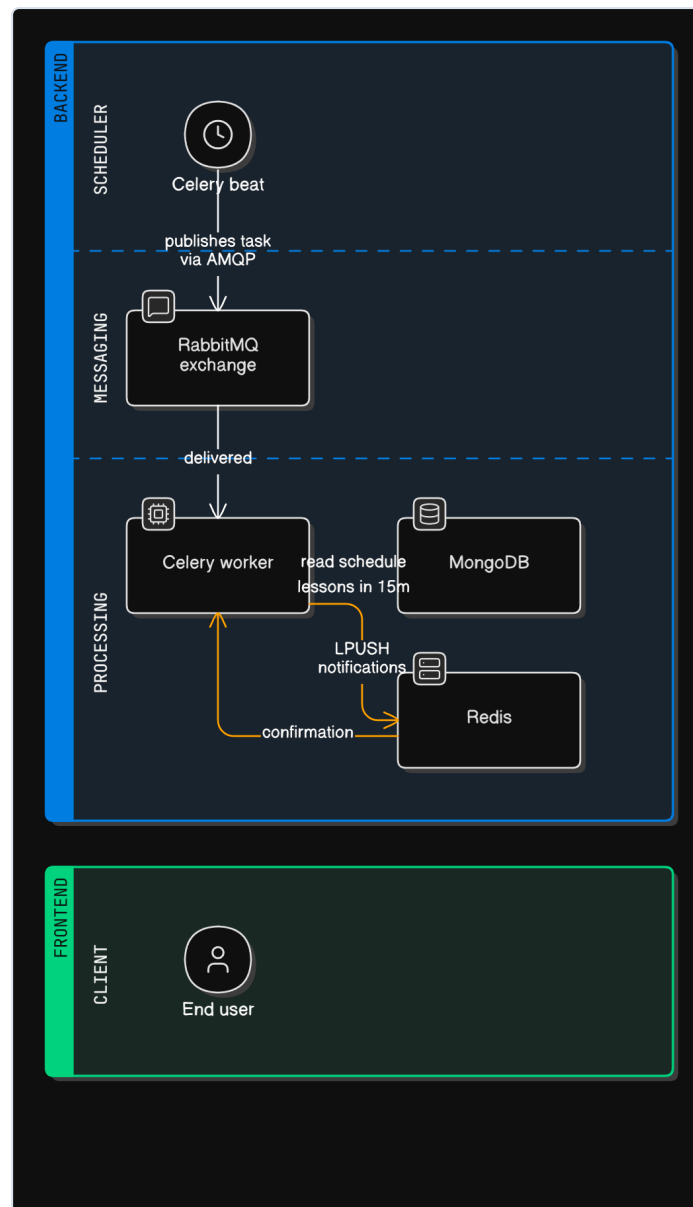


Figure 4 - BPMN: Celery beat (scheduler) publishes a task via AMQP → RabbitMQ exchange (messaging) → Celery worker (processing) reads Mongo schedule, LPUSHes notifications into Redis → end user polls GET /notifications from the frontend.

8. From-Scratch System Component (R11) - Token-Bucket Rate Limiter

8.1 What and why

REST endpoints that accept free-form text from authenticated users - specifically `POST /clubs/{id}/posts` - are obvious abuse targets. Without rate limiting, a single client can drown a club feed in seconds. We implemented a **token-bucket rate limiter** from scratch (no third-party library) and wired it as FastAPI middleware on the post endpoint.

Source lives at [server/auth/rate_limiter/bucket.py](#) and [server/auth/rate_limiter/middleware.py](#).

8.2 Algorithm

Each user × endpoint pair maps to a logical bucket of `capacity = 5` tokens. The bucket refills continuously at `refill_rate = 0.2` tokens/sec (one token every 5 s). A request consumes one token; when the bucket is empty the request is denied with `429 Too Many Requests` and a `retry_after_seconds` field telling the client when one token will be available again.

```
tokens(t) = min(capacity, tokens(t_last) + (t - t_last) * refill_rate)
if tokens(t) >= cost:
    tokens(t) -= cost                # request allowed
else:
    retry_after = (cost - tokens) / refill_rate
```

8.3 Design decisions and trade-offs

(a) Why token-bucket over alternatives.

- **Fixed-window counter** allows $2\times$ burst at a window boundary (a client posts 5 in second 59, 5 more in second 60).
- **Leaky bucket** smooths output rate but disallows bursts entirely - bad UX for users typing two posts in a minute.
- **Token bucket** allows controlled bursts up to `capacity` while still enforcing the long-run rate of `refill_rate`. That matches "a person can reasonably post 5 things, then needs to slow down" exactly.

(b) Why Redis (not in-process). The backend runs as **two replicas** behind `least_conn` Nginx (R8). An in-memory bucket on one replica would let a client multiply its quota by the number of replicas just by happening to land on different upstreams. Redis is a single source of truth.

(c) Why a Lua script. The take operation must read `(tokens, last_refill_timestamp)`, compute the new state, and write it back **atomically** - otherwise two near-simultaneous requests can both read the same old value, both decide they're allowed, and both write a `-1` state. Redis Lua scripts run server-side under the global single-threaded execution lock, giving us atomicity for free without external coordination.

The Lua script (see `bucket.py`, lines 44-78) returns `[allowed, tokens_remaining, retry_after]` and sets the bucket's TTL to `2 * time_to_full` so unused buckets evict themselves without a separate cleanup job.

8.4 Integration

`server/auth/rate_limiter/middleware.py` exposes `rate_limit(prefix, capacity, refill_rate)` as a FastAPI dependency. It extracts the JWT, resolves `user_id`, calls `bucket.take(user_id)`, and on denial:

1. inserts a row in `rate_limit_audit` (Postgres, async),
2. raises `HTTPException(429, ...)` with `Retry-After`.

The current production wiring is on `POST /clubs/{id}/posts` only, but the component is intentionally generic - adding it to any other endpoint is one line.

8.5 Limitations

- The bucket key is `user_id` only; an anonymous client (failed JWT) is rejected at the auth gate before reaching the limiter, so we do not currently bucket by IP. A real production deployment would add an IP bucket as a second line of defense.
- `time.time()` clock skew between replicas could theoretically cause the Lua script to receive a `now` slightly in the past; we mitigate with `math.max(0, now - ts)` inside the script. In practice both replicas run on the same droplet, so skew is zero.

8.6 Reference

Inspired by Kleppmann, Designing Data-Intensive Applications, ch. 8 ("The Trouble with Distributed Systems") - the bucket is a textbook example of moving coordination state into a shared store rather than attempting to gossip it between stateless replicas.

9. Infrastructure and Deployment (R8 + R9)

9.1 Hosting

The production deployment is a single **DigitalOcean droplet** (Ubuntu 22.04, 2 vCPU, 4 GB) reachable at **46.101.98.64**. The whole stack is brought up with:

```
ssh root@46.101.98.64
cd /opt/iut-community/server
docker compose up -d
```

The droplet exposes a single port (`8088`) to the public Internet via `ufw`; everything else (Postgres `5440`, Mongo `27017`, Redis `6379`, RabbitMQ `15672`, Airflow `8080`, Grafana `3000`) is bound but firewalled off and accessed only via SSH tunnels for ops.

9.2 Nginx gateway and load balancing (R8)

`server/gateway/nginx.conf` defines two upstream pools:

```

upstream auth_backend {
    least_conn;
    server backend-1:8000 max_fails=3 fail_timeout=10s;
    server backend-2:8000 max_fails=3 fail_timeout=10s;
}
upstream timetable_backend {
    server timetable:8001 max_fails=3 fail_timeout=10s;
}

```

`least_conn` picks the replica with fewer in-flight connections - appropriate because our request mix includes long-lived WebSocket connections that would skew a pure `round_robin` LB. WebSocket-specific headers (`Upgrade`, `Connection: upgrade`) are forwarded only on `/ws/`. TLS termination in production is wired through the same gateway by adding a `server { listen 443 ssl; ... }` block at deploy time (kept out of the checked-in `nginx.conf` to keep certs out of git).

9.3 Docker Compose (R9)

Single `docker-compose.yaml` orchestrates **15 services** in three networks:

- `backend-net` - backends, Postgres, Mongo, Redis, RabbitMQ, gateway, Celery, migration job
- `airflow-net` - airflow-postgres + 5 Airflow components
- `shared-net` - gateway + airflow-apiserver (so the gateway could route to the Airflow UI if we choose to expose it later)

Health checks on every long-lived service make the dependency arrows between services *enforceable*: `gateway` waits on `backend-1` / `backend-2` healthy, the backends wait on `migration` completed, `migration` waits on `backend-postgres` healthy, and so on. Named volumes (`backend-db-volume`, `mongo-db-volume`, `redis-volume`, `airflow-db-volume`) make state durable across `docker compose down/up`.

9.4 Environment variables

See `server/.env.example` for the full list, copied to `.env` at deploy time. Required: `DB_*`, `MONGO_*`, `SECRET_KEY`, `ALGORITHM`, `ACCESS_TOKEN_EXPIRE_MINUTES`, `FERNET_KEY`, `AIRFLOW_UID`, `RABBITMQ_USER`, `RABBITMQ_PASSWORD`.

10. Observability (R12)

Both FastAPI services bootstrap the **OpenTelemetry SDK** in `telemetry.py`:

- `TracerProvider` with FastAPI + SQLAlchemy + Requests instrumentations (automatically wraps every HTTP handler, every SQL query, and every outbound HTTP call into spans).
- `MeterProvider` exporting RED metrics (Rate, Errors, Duration).
- `LoggerProvider` shipping `LogRecord`s out of the standard library `logging` module.

All three exporters target the same OTLP/HTTP endpoint (`http://observability:4318`), which is the **Grafana LGTM** all-in-one container (Loki + Grafana + Tempo + Prometheus + OpenTelemetry collector, shipped as a single image - `grafana/otel-lgtm`). The Grafana UI is at **`http://46.101.98.64:3000`** (`admin / admin`, behind the firewall).

10.1 What we collect

Signal	Source	Backend	Use case
Traces	OpenTelemetry FastAPI + SQLAlchemy auto-instrumentation	Tempo	"Why was this <code>/dashboard</code> slow?" → drill into the 4-query GraphQL resolver trace.
Logs	OTel handler attached to root logger	Loki	"Show me every rate-limit denial in the last hour" - filtered by <code>severity = WARN AND endpoint = /clubs/.+/posts</code> .
Metrics	RED + Postgres + Redis exporters	Prometheus	Dashboards on req/s, error rate, p95 latency, Redis cache hit ratio.

10.2 Sample correlated drill-down (UC-2)

A user clicks "My dashboard". The frontend hits `POST /graphql`:

1. **Trace** - single root span `POST /graphql` (gateway) → `POST /graphql` (backend-1) → 4 sibling spans (`SELECT users`, `SELECT clubs`, `SELECT bookings`, `SELECT assignments`) → return. Total p95 = 38 ms warm, 172 ms cold. Visible in Tempo with `trace_id` linked from the Loki log query.
2. **Logs** - the access log line emitted by the FastAPI handler carries the trace ID as `trace_id=<...>`, so the same query in Loki shows the request log inline with the trace.

3. **Metrics** - `http_server_duration_milliseconds_bucket{path="/graphql"}` shows the histogram bin distribution; we use a Grafana panel that plots p95 over time.

Live Grafana dashboards are reachable from **`http://46.101.98.64:3000`** (admin/admin, over SSH tunnel). The traces / logs / metrics correlation above can be reproduced live from a single `POST /graphql` request against the deployed URL.

10.3 Live evidence from the deployed stack

The six screenshots below are captured from the running Grafana instance on the production droplet (`46.101.98.64:3000`), inside the **Explore** → **Logs (Loki)** view. They show that both FastAPI services are actually streaming OTel logs through OTLP/HTTP into Loki, with all the OTel resource attributes preserved as queryable fields.

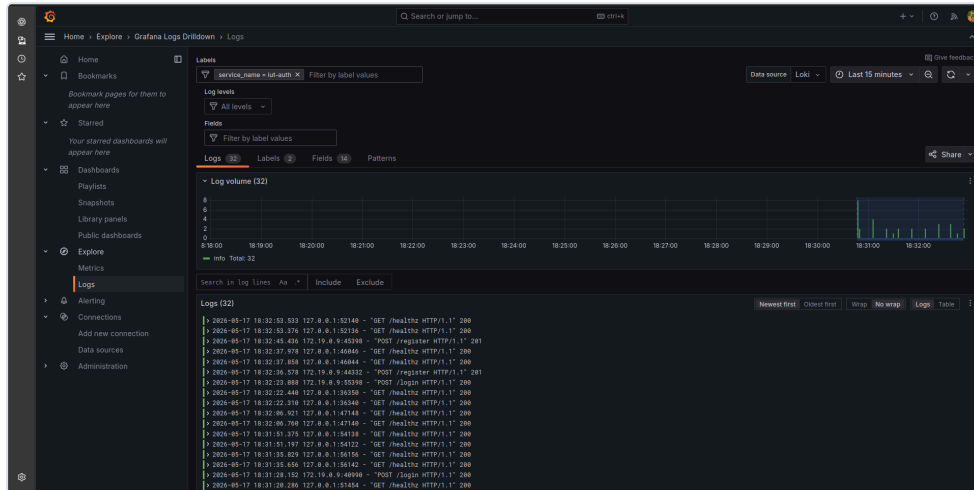


Figure 6 - Loki Explore filtered by `service_name = iut-auth`. The log-volume sparkline (top) shows a sustained stream of structured access log lines from the auth service; the bottom panel lists the raw log records (32 in the last 15 minutes) emitted by the FastAPI OTEL *LoggerProvider*.

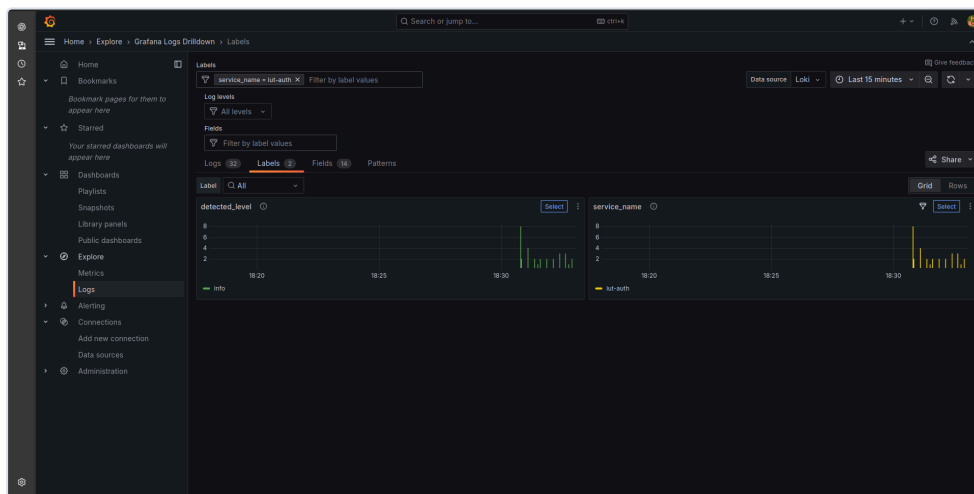


Figure 7 - Same query, **Labels** tab. Two indexed labels are exposed by the OTEL resource: `detected_level = info` and `service_name = iut-auth`. The per-label histograms confirm the auth service is logging continuously at INFO level over the selected window.

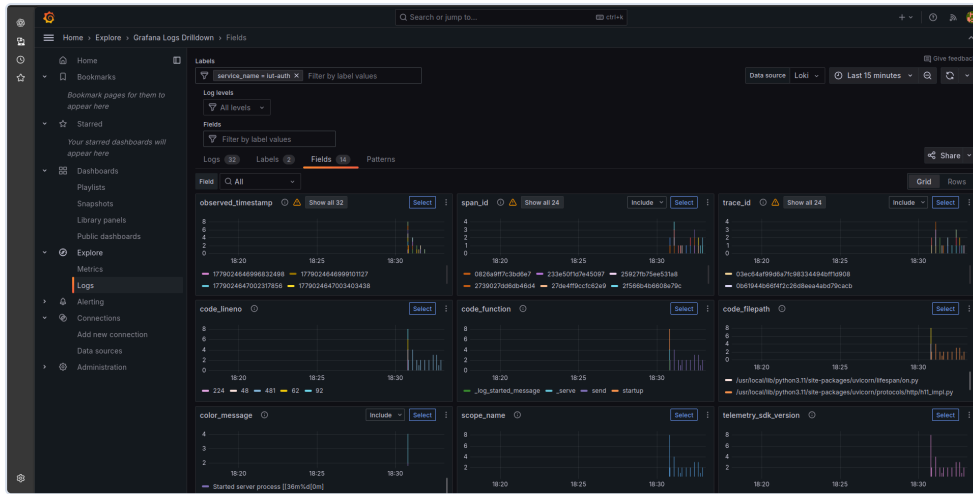


Figure 8 - **Fields** tab for the same query. Loki has automatically extracted 14 OTEL attributes from each log line, including `trace_id` and `span_id` (which is what enables the Tempo ↔ Loki drill-down described in §10.2), plus source-code context (`code_filepath`, `code_function`, `code_lineno`) and SDK metadata (`telemetry_sdk_version`).

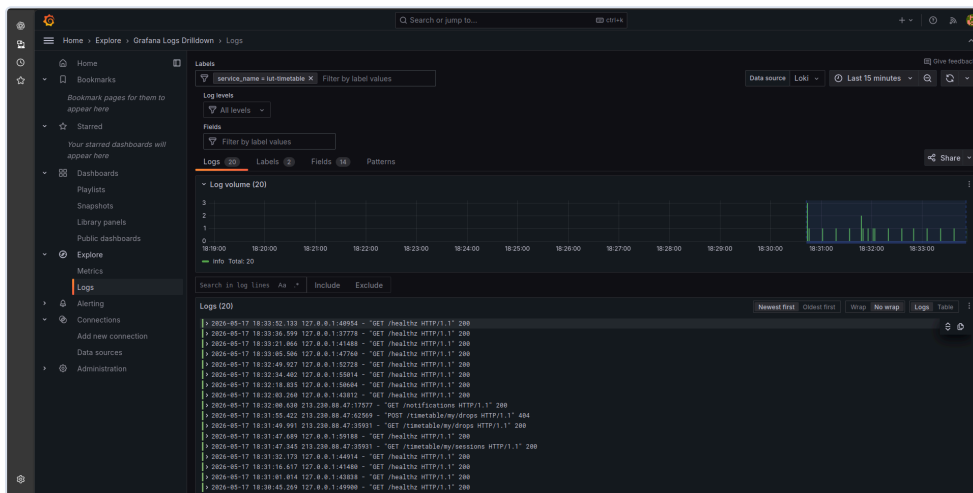


Figure 9 - The same Explore view filtered by `service_name = iut-timetable`. The timetable FastAPI service (the second backend, exposing schedule queries and Celery-triggered reminders) is independently shipping its own log stream into the same Loki instance, demonstrating that the multi-service OTEL pipeline is wired correctly.

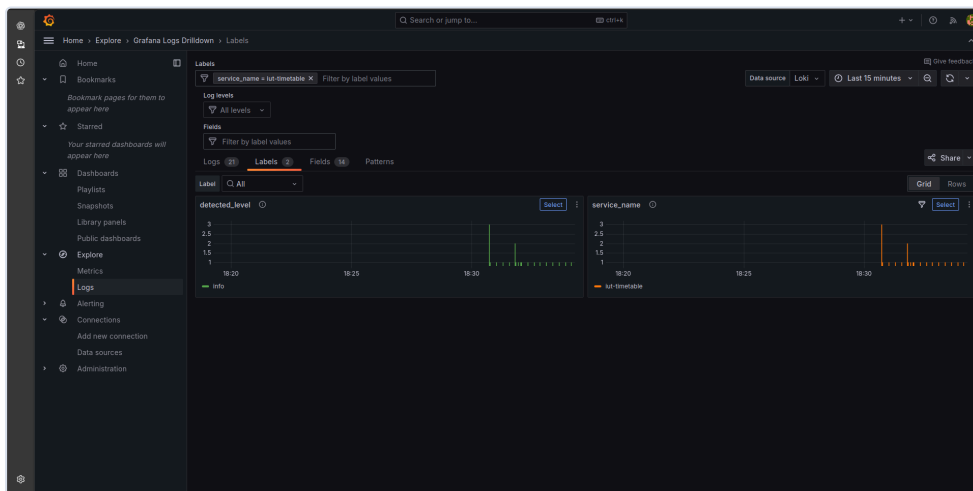


Figure 10 - **Labels** tab for `iut-timetable`. 21 log lines in the 15-minute window, again with `detected_level = info` and `service_name = iut-timetable` as queryable labels. The fact that both services appear with distinct `service_name` values proves the resource attribute set in `telemetry.py` is propagating end-to-end.

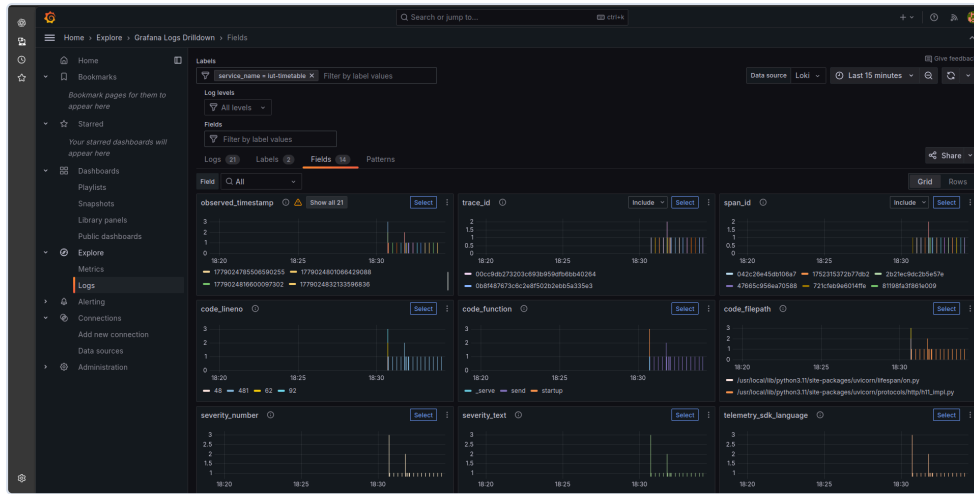


Figure 11 - **Fields** tab for the timetable service. Same 14 Otel attributes are extracted, including `severity_number` / `severity_text` (so we can build LogQL queries like `{service_name="iut-timetable"} | severity_text="ERROR"` for alerting), and `trace_id` linking each log line back to its originating Tempo span.

11. Testing and Known Limitations

11.1 Tested manually

- **Auth** - register / login / `/me` round-trips against the deployed URL.
- **Rate limiter** - for `i` in `{1..10}`; do `curl -X POST ... /clubs/X/posts`; done produces `5 × 201` followed by `429` with monotonically increasing `retry_after_seconds` - exactly what the algorithm predicts.
- **WebSocket chat** - opened two browser tabs as different users in the same club, verified backlog replay + live broadcast.
- **GraphQL dashboard** - single round-trip returns all four resources.
- **Airflow** - manually triggered `nightly_analytics` from the Airflow UI, verified `room_usage_daily` rows appear.
- **Gateway LB** - for `i` in `{1..20}`; do `curl .../healthz`; done distributes across `backend-1` and `backend-2` per the Nginx access log.
- **Migrations** - `docker compose down -v && docker compose up -d` brings up a clean DB and applies seed.

11.2 Known limitations (intentional, would address with more time)

#	Limitation	Why
L-1	TLS terminates plaintext on <code>:8088</code> ; production would need Let's Encrypt + a 443 listener.	Out of scope for the v1.0 internal demo; the gateway is configured to accept the listener block when the certs land.
L-2	Rate-limiter keys only by <code>user_id</code> , not IP.	Unauthenticated traffic is rejected at the JWT gate, so IP-keying would only add value if we ever exposed an anonymous endpoint.
L-3	Chat WebSocket <code>ConnectionManager</code> is per-replica.	A user with two browser tabs on different replicas sees both messages because the Mongo persist round-trip + Redis backlog re-fetch already deliver - but cross-replica live broadcast would need a Redis pub/sub fan-out for true symmetry.
L-4	No automated test suite.	Time-boxed for v1.0; manual smoke tests above.
L-5	Grafana dashboards are imported from the LGTM defaults; we have not curated a bespoke "IUT Community" dashboard.	The default RED dashboard is good enough for the demo.

12. Team Contribution Table

Member	Student ID	Primary ownership	Notable commits / artefacts
Kamoliddin Sharopov (lead)	U2310245	DevOps, infrastructure, observability	<code>docker-compose.yaml</code> , <code>gateway/nginx.conf</code> , <code>observability/docker-compose.yaml</code> , droplet deployment, OTel bootstrap
Sardorbek Suyunov	U2310262	Database architect, ER diagram, migrations	<code>models.py</code> , <code>db/versions/a1b2c3d4e5f6_add_community_tables.py</code> , <code>db/seed.py</code> , ER diagram + report oversight
Nurmukhammad Shomahmudov	U2310249	Backend - REST + auth + Postgres	<code>auth/main.py</code> , <code>auth/routers/clubs.py</code> , <code>auth/routers/posts.py</code> , <code>auth/routers/dashboard.py</code> , <code>auth/jwt.py</code>
Farrukh Sattorov	U2310237	Backend - WebSockets + Celery + R11	<code>auth/ws/chat.py</code> , <code>auth/ws/notifications.py</code> , <code>auth/rate_limiter/</code> , <code>time_table_backend/celery_app.py</code>
Bekhruz To'ymurodov	U2310268	Frontend (React + Vite + shadcn/ui)	<code>frontend/src/app/*</code> , <code>frontend/src/app/lib/api.ts</code> , <code>LoginPage</code> , <code>OverviewPage</code> , <code>CommunityPage</code> , <code>ChatPanel</code>
Bobur Rustamov	U2310221	Documentation	<code>README.md</code> , <code>CHANGELOG.md</code> , this report, BPMN diagrams

Commit history (`git log --author=... --oneline | wc -l`) is the source of truth and is reviewable on the GitHub repos linked in §0.

13. References

1. Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly, 2017.
2. Xu, A. *System Design Interview - An Insider's Guide*. ByteByteGo, 2020.
3. FastAPI documentation - <https://fastapi.tiangolo.com>
4. Nginx - `least_conn` upstream balancer - https://nginx.org/en/docs/http/nginx_http_upstream_module.html
5. Redis Lua scripting - <https://redis.io/docs/latest/develop/interact/programmability/eval-intro/>
6. Apache Airflow - <https://airflow.apache.org/docs/>
7. OpenTelemetry Python - <https://opentelemetry.io/docs/languages/python/>
8. Grafana LGTM all-in-one - <https://github.com/grafana/docker-otel-lgtm>
9. PostgreSQL full-text search - <https://www.postgresql.org/docs/current/textsearch.html>
10. `arthur-zhang/letsddia-go` - DDIA building blocks (for R11 inspiration).